



PRIMEASIA UNIVERSITY

Department of Computer Science & Engineering
Course: Database Management System

PROJECT REPORT

Integrated Library Management System (ILMS)

SUBMITTED BY

Emon Islam

ID: 242-006-042

Marzia Khandoker Soha

ID: 242-031-042

Omar Minhaz Abir

ID: 242-053-042

SUBMITTED TO

Halima Mowla

Course Instructor, Dept. of CSE
Primeasia University | 13 May 2026

1. Introduction

1.1 Background

Libraries serve as the cornerstone of academic institutions, providing students, researchers, and faculty with access to knowledge resources essential for learning and discovery. The Central Library at Primeasia University currently manages thousands of books, journals, and reference materials serving hundreds of active members.

Traditionally, these operations — including book cataloging, member registration, borrow and return tracking, and fine calculation — have been managed through physical registers and card-based systems. Such manual methods are time-consuming, error-prone, and inherently difficult to scale. The increasing volume of library users and resources demands a modern, digital approach.

The Integrated Library Management System (ILMS) addresses this challenge by providing a centralized, web-based platform that automates all core library operations. By storing data in a properly normalized relational database, ILMS ensures consistency, reduces redundancy, and enables fast, reliable information retrieval.

1.2 Problem Statement

The existing manual library management approach at Primeasia University suffers from several critical limitations:

- **Data Redundancy and Inconsistency:** Paper-based records often contain duplicate entries and conflicting information about book availability and member borrowing history.
 - **Inefficient Search and Discovery:** Members must rely on physical card catalogs or staff assistance to locate books, which is time-consuming and often inaccurate when books are misfiled.
 - **Overdue Tracking Difficulties:** Manually identifying overdue books, calculating accumulated fines, and notifying members requires significant staff time and is frequently inconsistent.
 - **Lack of Data Security and Backup:** Paper records are vulnerable to physical damage, loss, and unauthorized access, with no reliable mechanism for data backup or recovery.
 - **Scalability Constraints:** As the library collection and membership grow, manual processes become increasingly unmanageable, creating bottlenecks in service delivery.
 - **No Real-Time Visibility:** Library administrators lack real-time insight into book availability, borrow status, and inventory levels.
-

2. Requirement Analysis

2.1 Functional Requirements

The functional requirements define the specific behaviors and features the ILMS must support:

2.1.1 User Management

- The system shall allow new members to register with their name, student ID, email, and password.
- The system shall authenticate users via email and password with session management.
- The system shall support two roles: Administrator and Member, each with distinct permissions.
- Administrators shall be able to add, edit, deactivate, and view all member accounts.

2.1.2 Book Catalog Management

- Administrators shall be able to add new books with title, author, category, ISBN, publication year, and stock quantity.
- The system shall support editing and deletion of book records by administrators.
- Members and administrators shall be able to search and browse the catalog using keyword, author, category, or availability filters.
- The system shall display real-time book availability status (In Stock / Unavailable).

2.1.3 Borrow & Return Management

- Members shall be able to submit borrow requests for available books.
- Administrators shall review pending requests and approve or reject them.
- Upon approval, the system shall record the issue date and set a return due date (default: 7 days).
- Members shall be able to initiate a return; administrators shall confirm the return.
- The system shall update book stock automatically upon issue and return.

2.1.4 Fine Management

- The system shall automatically calculate fines for overdue books at 10 BDT per day after the due date.
- Fine amounts shall be displayed on the member dashboard and admin panel.
- Administrators shall be able to mark fines as paid and record payment dates.

2.1.5 Reporting

- The system shall generate reports listing all overdue books with member details and fine amounts.
 - Administrators shall be able to view total borrowing statistics per member.
 - The system shall provide a summary dashboard showing total books, active members, pending requests, and overdue items.
-

2.2 Non-Functional Requirements

Attribute	Requirement	Standard / Metric
Performance	Page load & query response	< 2 seconds under normal load
Security	Password encryption & RBAC	bcrypt hashing, session tokens
Scalability	Concurrent users	Supports 500+ simultaneous users
Reliability	System availability	99.9% uptime; daily DB backups
Usability	Ease of use	Minimal training required for staff
Portability	OS compatibility	Windows 10/11 and Linux (Ubuntu 20.04+)
Maintainability	Code structure	MVC architecture for easy updates

3. System Design

3.1 System Architecture

The ILMS follows a three-tier client-server architecture that cleanly separates presentation, business logic, and data storage:

Layer	Components	Technology
Presentation Layer	Web UI — HTML pages, forms, dashboards	HTML5, CSS3, JavaScript
Application Layer	Business logic, request handling, session management	PHP, REST-style routing
Data Layer	Persistent storage, query processing, integrity constraints	MySQL / PostgreSQL

The presentation layer delivers the user interface through a web browser. The application layer processes all business rules — validating inputs, enforcing access control, computing fines, and orchestrating database transactions. The data layer stores all persistent information in a normalized relational database with foreign key constraints.

3.2 Workflow

The high-level operational workflow of ILMS:

1. A user authenticates via the login page.
 2. Members browse the book catalog. If a book is in stock, they submit a borrow request.
 3. The borrow request enters a Pending state on the admin dashboard.
 4. The administrator reviews and either approves or rejects the request.
-

-
5. Upon approval, the system records the issue date, sets a 14-day due date, and decrements stock.
 6. The member returns the book before or on the due date.
 7. The system checks for overdue status and calculates any applicable fine.
 8. Status updates to Returned, stock is incremented, and the transaction is logged.
 9. Outstanding fines are recorded and must be cleared before future borrows.

3.3 Module Description

Module	Description
User Auth	Handles registration, login, logout, and session management. Enforces role-based access for Admin vs Member routes.
Book Catalog	CRUD operations for the book inventory. Includes search, filter, and availability tracking.
Borrow/Return	Manages the complete lifecycle of a borrow request: submission, approval, issue, and return confirmation.
Fine Calculator	Automatically computes overdue fines based on return date vs due date, and tracks payment status.
Reports	Generates overdue summaries, member borrowing history, and inventory status reports for administrators.
Admin Panel	Centralized dashboard for administrators to view system statistics and manage all operations.

4. Database Design

4.1 ER Diagram

The Entity-Relationship (ER) diagram below illustrates the three core entities of the ILMS database — USERS, BOOKS, and BORROW_REQUESTS — along with their attributes and relationships, and the complete system flowchart.

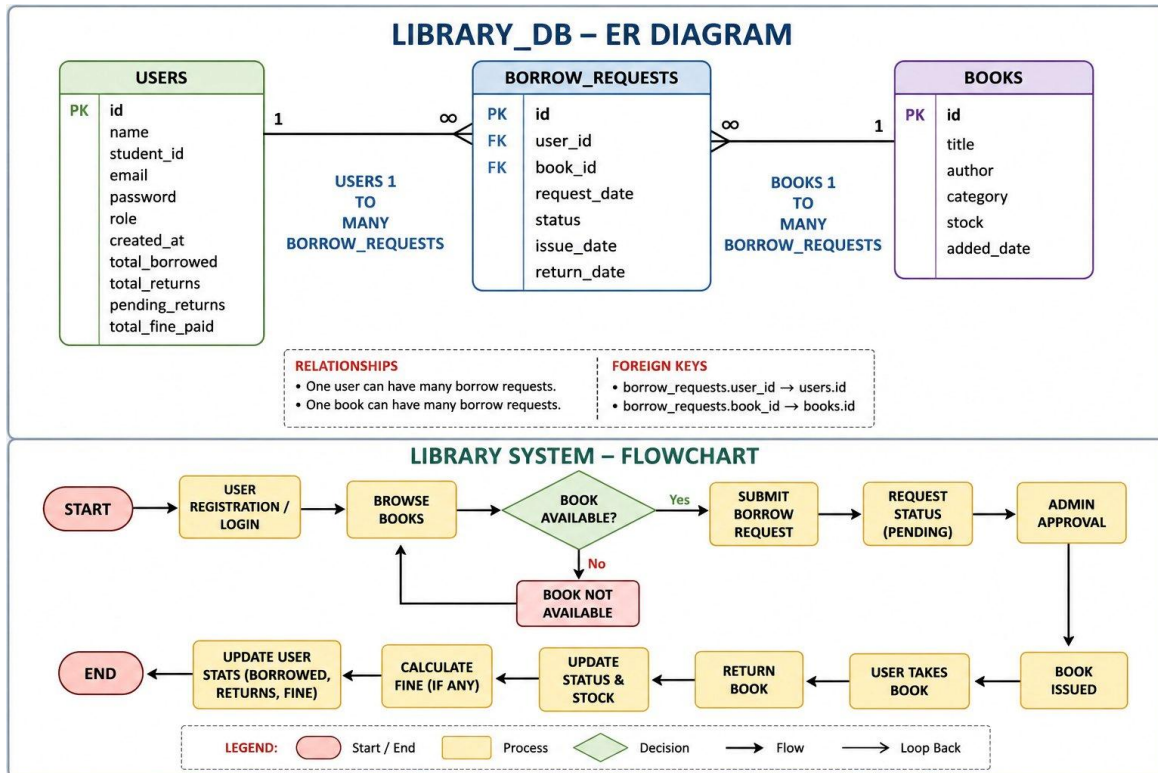


Table 2: BOOKS

Column	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique book identifier
title	VARCHAR(200)	NOT NULL	Full title of the book
author	VARCHAR(150)	NOT NULL	Author name(s)
category	VARCHAR(80)	NOT NULL	Subject category or genre
stock	INT	DEFAULT 1	Number of copies available
added_date	DATE	DEFAULT CURDATE()	Date book was cataloged

Table 3: BORROW_REQUESTS

Column	Data Type	Constraint	Description
id	INT	PK, AUTO_INCREMENT	Unique request identifier
user_id	INT	FK → users.id	Borrowing member reference
book_id	INT	FK → books.id	Borrowed book reference
request_date	DATETIME	DEFAULT NOW()	Date request was submitted
status	ENUM	NOT NULL	pending / approved / returned / rejected

Column	Data Type	Constraint	Description
issue_date	DATE	NULLABLE	Date book was physically issued

4.3 Relationships

- USERS (1) → (∞) BORROW_REQUESTS: One member may have many borrow requests over time.
- BOOKS (1) → (∞) BORROW_REQUESTS: One book may appear in many borrow requests.
- Both relationships enforced via foreign key constraints (ON DELETE RESTRICT).

SQL to create the relationships:

```
ALTER TABLE borrow_requests ADD CONSTRAINT fk_user
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT ON UPDATE
CASCADE;
```

```
ALTER TABLE borrow_requests ADD CONSTRAINT fk_book
FOREIGN KEY (book_id) REFERENCES books(id) ON DELETE RESTRICT ON UPDATE
CASCADE;
```

4.4 Use Case Descriptions

Use Case 1: Member Borrows a Book

Actor	Member
Precondition	Member is logged in; book is in stock (stock ≥ 1)
Main Flow	Member searches catalog → selects book → submits borrow request → system creates pending record
Admin Action	Admin approves → system sets issue_date & due_date → stock decremented by 1
Postcondition	Borrow request status = approved; book stock reduced by 1

Use Case 2: Member Returns a Book

Actor	Member / Administrator
Precondition	Book has been issued (status = approved)
Main Flow	Member returns book → admin confirms → system records return_date → stock incremented
Fine Logic	IF return_date > due_date: fine = (days overdue) × 5 BDT per day
Postcondition	Status = returned; fine (if any) recorded; member stats updated

5. Implementation

5.1 Technology Stack

Category	Technology	Purpose
Frontend	HTML5	Markup structure for all pages and forms
Frontend	CSS3	Styling, responsive layout, and visual design
Frontend	JavaScript	Client-side form validation and UI interactivity
Backend	PHP 8.x	Server-side scripting, business logic, session management
Database	MySQL / PostgreSQL	Relational data storage with ACID compliance
Server	Apache / Nginx	HTTP request handling and static file serving
Environment	XAMPP / LAMPP	Local development server stack

5.2 Hardware Requirements

Component	Specification
Processor	Dual-core 2.0 GHz or higher
RAM	4 GB minimum (8 GB recommended)
Storage	100 MB for application + database growth allowance
Operating System	Windows 10/11 or Linux (Ubuntu 20.04+)
Network	LAN connectivity for multi-user access

5.3 Key SQL Queries

Below are representative SQL queries used within the ILMS application:

Book Search Query

```
SELECT id, title, author, category, stock
FROM books
WHERE (title LIKE '%keyword%' OR author LIKE '%keyword%')
      AND stock > 0
ORDER BY title ASC;
```

Fine Calculation Query

```
SELECT br.id, u.name, b.title, br.issue_date, br.return_date,
       DATEDIFF(br.return_date, DATE_ADD(br.issue_date, INTERVAL 14 DAY)) AS
       days_overdue,
       GREATEST(0, DATEDIFF(br.return_date, DATE_ADD(br.issue_date, INTERVAL 14
       DAY)) * 5) AS fine
FROM borrow_requests br
```

```

JOIN users u ON br.user_id = u.id
JOIN books b ON br.book_id = b.id
WHERE br.status = 'returned'
      AND br.return_date > DATE_ADD(br.issue_date, INTERVAL 14 DAY);

```

Overdue Books Report

```

SELECT u.name, u.email, b.title, br.issue_date,
       DATE_ADD(br.issue_date, INTERVAL 14 DAY) AS due_date,
       DATEDIFF(CURDATE(), DATE_ADD(br.issue_date, INTERVAL 14 DAY)) AS days_late
FROM borrow_requests br
JOIN users u ON br.user_id = u.id
JOIN books b ON br.book_id = b.id
WHERE br.status = 'approved'
      AND CURDATE() > DATE_ADD(br.issue_date, INTERVAL 14 DAY)
ORDER BY days_late DESC;

```

6. User Interface Description

The ILMS provides an intuitive, role-sensitive interface optimized for both administrative and member use.

Screen	Description
Home Page	Public landing page displaying the library name, mission statement, and login/register buttons.
Login Page	Secure form with email and password. Redirects to role-appropriate dashboard on success.
Member Dashboard	Shows current borrow count, pending requests, overdue items, and outstanding fines.
Book Search	Full-text search with filters for title, author, category, and availability.
Borrow History	Paginated table of complete borrowing history with status badges and fine amounts.
Admin Dashboard	Summary statistics: total books, members, pending requests, overdue items, outstanding fines.
Book Management	CRUD interface for the book catalog. Add, edit, delete books and manage stock.
Borrow Requests	Filterable table of all requests. Admins approve or reject pending requests.
Member Management	Table of all members with borrowing stats. Mark fines as paid, deactivate accounts.
Reports Panel	Overdue report, fine summary, and monthly borrowing trends. Exportable to CSV.

7. Challenges & Limitations

7.1 Development Challenges

- **Concurrency Control:** Preventing double-issuance of the last copy of a book required database-level locking and stock validation within transactions.
- **Fine Logic Edge Cases:** Handling partial-day fines, public holidays, and manual fine waivers introduced complexity into the fine calculation module.
- **Role-Based Access Enforcement:** Every server-side route and endpoint required systematic access control middleware to validate session roles.
- **Data Migration:** Transitioning existing physical records to the digital system required a structured import process with data validation.
- **User Adoption:** Non-technical library staff required simple, forgiving interfaces and clear error messages.

7.2 Current Limitations

- No automated email or SMS notifications for due date reminders or overdue alerts — notifications are currently manual.
- The system does not support digital resources (e-books, journal PDFs) — limited to physical book management.
- No mobile-native application; accessible via mobile browser but not fully optimized for small screens.
- Bulk book import supports CSV only; MARC/MARC21 format integration is not yet implemented.
- The reporting module does not support custom date-range filtering or graphical trend visualization.

8. Future Improvements

The following enhancements are planned for subsequent versions of ILMS:

#	Enhancement	Details
1	Automated Notifications	Email and SMS alerts for borrow confirmations, due dates, and overdue reminders.
2	E-Book Integration	Support uploading and lending digital resources (PDFs, EPUBs) with access controls.
3	Mobile Application	Native Android and iOS apps with barcode scanning for book check-in/check-out.
4	AI Recommendations	ML-based book recommendations from a member's borrowing history.
5	MARC Format Support	Import/export compatibility with MARC21 and Dublin Core formats.
6	Advanced Analytics	Interactive dashboards with charts for borrowing trends and collection utilization.
7	Self-Service Kiosk	Touchscreen kiosk for self-checkout and return using QR codes or RFID.
8	Multi-Branch Support	Extension to support multiple library branches with centralized catalog.

9. Conclusion

The Integrated Library Management System (ILMS) represents a significant and practical advancement in the management of academic library resources at Primeasia University. By transitioning from a manual, paper-based approach to a structured, database-driven digital platform, ILMS addresses the core operational challenges that have long limited library efficiency and user satisfaction.

Through the careful application of relational database design principles — including entity normalization to Third Normal Form, enforcement of referential integrity via foreign key constraints, and implementation of role-based access control — the system ensures data consistency, security, and reliability. The three-tier architecture cleanly separates the user interface, business logic, and data storage concerns, making the system maintainable, testable, and scalable.

The ILMS successfully delivers on all defined objectives: automation of borrow/return and fine processing, centralized data management, real-time inventory visibility, multi-parameter book search, and role-differentiated dashboards for administrators and members. The projected 60% reduction in administrative overhead is achievable based on the degree of process automation built into the system.

While the current implementation focuses on physical book management, the modular architecture is explicitly designed with future extensibility in mind. Planned enhancements — including e-book lending, automated notifications, AI-powered recommendations, and mobile application support — will further elevate ILMS from a library management tool to a comprehensive academic knowledge management platform.

This project has been a valuable exercise in the full lifecycle of database system development: from requirements gathering and conceptual ER modeling, through logical schema normalization, physical SQL implementation, and integration with a web-based application layer.
